



Redis Enterprise Software REST API quick start

Redis Enterprise Software includes a REST API that allows you to automate certain tasks. This article shows you how to send a request to the Redis Enterprise Software REST API.

Fundamentals

No matter which method you use to send API requests, there are a few common concepts to remember.

Type	Description
Authentication	Use Basic Auth with your cluster username (email) and password
Ports	All calls are made to port 9443 by default
Versions	Specify the version in the request URI
Headers	Accept and Content-Type should be application/json
Response types and error codes	A response of 200 OK means success; otherwise, the request failed due to an error

For more information, see [Redis Enterprise Software REST API](#).

cURL example requests

[cURL](#) is a command-line tool that allows you to send HTTP requests from a terminal.

You can use the following options to build a cURL request:

Option	Description
-X	Method (GET, PUT, PATCH, POST, or DELETE)
-H	Request header, can be specified multiple times
-u	Username and password information
-d	JSON data for PUT or POST requests
-F	Form data for PUT or POST requests, such as for the POST /v1/modules or POST /v2/modules endpoint
-k	Turn off SSL verification
-i	Show headers and status code as well as the response body

See the [cURL documentation](#) for more information.

GET request

Use the following cURL command to get a list of databases with the [GET /v1/bdbs/](#) endpoint.

```
$ curl -X GET -H "accept: application/json" \
      -u "[username]:[password]" \
      https://[host][:port]/v1/bdbs -k -i
```

```
HTTP/1.1 200 OK
server: envoy
date: Tue, 14 Jun 2022 19:24:30 GMT
content-type: application/json
content-length: 2833
cluster-state-id: 42
x-envoy-upstream-service-time: 25
```

```
[
  {
    ...
    "name": "tr01",
    ...
    "uid": 1,
    "version": "6.0.16",
    "wait_command": true
  }
]
```

In the response body, the `uid` is the database ID. You can use the database ID to view or update the database using the API.

For more information about the fields returned by [GET /v1/bdbs/](#), see the [bdbs object](#).

PUT request

Once you have the database ID, you can use [PUT /v1/bdbs/](#) to update the configuration of the database.

For example, you can pass the database `uid 1` as a URL parameter and use the `-d` option to specify the new name when you send the request. This changes the database's name from `tr01` to `database1`:

```
$ curl -X PUT -H "accept: application/json" \
      -H "content-type: application/json" \
      -u "cameron.bates@redis.com:test123" \
      https://[host]:[port]/v1/bdbs/1 \
      -d '{ "name": "database1" }' -k -i
```

```
HTTP/1.1 200 OK
server: envoy
date: Tue, 14 Jun 2022 20:00:25 GMT
content-type: application/json
content-length: 2933
cluster-state-id: 43
x-envoy-upstream-service-time: 159
```

```
{
  ...
  "name" : "database1",
  ...
  "uid" : 1,
  "version" : "6.0.16",
  "wait_command" : true
}
```

For more information about the fields you can update with [PUT /v1/bdbs/](#), see the [bdbs object](#).

Client examples

You can also use client libraries to make API requests in your preferred language.

To follow these examples, you need:

- A [Redis Enterprise Software](#) node
- Python 3 and the [requests](#) Python library
- [node.js](#) and [node-fetch](#)

Python

```

import json
import requests

# Required connection information - replace with your host, port, username, and password
host = "[host]"
port = "[port]"
username = "[username]"
password = "[password]"

# Get the list of databases using GET /v1/bdbs
bdbs_uri = "https://{}/{}/v1/bdbs".format(host, port)

print("GET {}".format(bdbs_uri))
get_resp = requests.get(bdbs_uri,
                        auth = (username, password),
                        headers = { "accept" : "application/json" },
                        verify = False)

print("{} {}".format(get_resp.status_code, get_resp.reason))
for header in get_resp.headers.keys():
    print("{}: {}".format(header, get_resp.headers[header]))

print("\n" + json.dumps(get_resp.json(), indent=4))

# Rename all databases using PUT /v1/bdbs
for bdb in get_resp.json():
    uid = bdb["uid"] # Get the database ID from the JSON response

    put_uri = "{}/{}/".format(bdbs_uri, uid)
    new_name = "database{}".format(uid)
    put_data = { "name" : new_name }

    print("PUT {}".format(put_uri, json.dumps(put_data)))

    put_resp = requests.put(put_uri,
                           data = json.dumps(put_data),
                           auth = (username, password),
                           headers = { "content-type" : "application/json" },
                           verify = False)

    print("{} {}".format(put_resp.status_code, put_resp.reason))
    for header in put_resp.headers.keys():
        print("{}: {}".format(header, put_resp.headers[header]))

    print("\n" + json.dumps(put_resp.json(), indent=4))

```

See the [Python requests library documentation](#) for more information.

Output

```
$ python rs_api.py
python rs_api.py
GET https://[host]:[port]/v1/bdbs
InsecureRequestWarning: Unverified HTTPS request is being made to host '[host]'.
Adding certificate verification is strongly advised.
See: https://urllib3.readthedocs.io/en/1.26.x/advanced-usage.html#ssl-warnings
warnings.warn(
200 OK
server: envoy
date: Wed, 15 Jun 2022 15:49:43 GMT
content-type: application/json
content-length: 2832
cluster-state-id: 89
x-envoy-upstream-service-time: 27

[
  {
    ...
    "name": "tr01",
    ...
    "uid": 1,
    "version": "6.0.16",
    "wait_command": true
  }
]
```

```
PUT https://[host]:[port]/v1/bdbs/1 {"name": "database1"}
InsecureRequestWarning: Unverified HTTPS request is being made to host '[host]'.
Adding certificate verification is strongly advised.
See: https://urllib3.readthedocs.io/en/1.26.x/advanced-usage.html#ssl-warnings
warnings.warn(
200 OK
server: envoy
date: Wed, 15 Jun 2022 15:49:43 GMT
content-type: application/json
content-length: 2933
cluster-state-id: 90
x-envoy-upstream-service-time: 128

{
  ...
  "name" : "database1",
  ...
  "uid" : 1,
  "version" : "6.0.16",
  "wait_command" : true
}
```

node.js

```

import fetch, { Headers } from 'node-fetch';
import * as https from 'https';

const HOST = '[host]';
const PORT = '[port]';
const USERNAME = '[username]';
const PASSWORD = '[password]';

// Get the list of databases using GET /v1/bdbs
const BDBS_URI = `https://${HOST}:${PORT}/v1/bdbs`;
const USER_CREDENTIALS = Buffer.from(`${USERNAME}:${PASSWORD}`).toString('base64');
const AUTH_HEADER = `Basic ${USER_CREDENTIALS}`;

console.log(`GET ${BDBS_URI}`);

const HTTPS_AGENT = new https.Agent({
  rejectUnauthorized: false
});

const response = await fetch(BDBS_URI, {
  method: 'GET',
  headers: {
    'Accept': 'application/json',
    'Authorization': AUTH_HEADER
  },
  agent: HTTPS_AGENT
});

const responseObject = await response.json();
console.log(`${response.status}: ${response.statusText}`);
console.log(responseObject);

// Rename all databases using PUT /v1/bdbs
for (const database of responseObject) {
  const DATABASE_URI = `${BDBS_URI}/${database.uid}`;
  const new_name = `database${database.uid}`;

  console.log(`PUT ${DATABASE_URI}`);

  const response = await fetch(DATABASE_URI, {
    method: 'PUT',
    headers: {
      'Authorization': AUTH_HEADER,
      'Content-Type': 'application/json'
    },
    body: JSON.stringify({
      'name': new_name
    }),
    agent: HTTPS_AGENT
  });

  console.log(`${response.status}: ${response.statusText}`);
  console.log(await response.json());
}

```

See the [node-fetch documentation](#) for more info.

Output

```
$ node rs_api.js
GET https://[host]:[port]/v1/bdbs
200: OK
[
  {
    ...
    "name": "tr01",
    ...
    "slave_ha" : false,
    ...
    "uid": 1,
    "version": "6.0.16",
    "wait_command": true
  }
]
PUT https://[host]:[port]/v1/bdbs/1
200: OK
{
  ...
  "name" : "tr01",
  ...
  "slave_ha" : true,
  ...
  "uid" : 1,
  "version" : "6.0.16",
  "wait_command" : true
}
```

More info

- [Redis Enterprise Software REST API](#)
- [Redis Enterprise Software REST API requests](#)

Updated: July 5, 2022